

SEGMENTACIÓN MEDIANTE THRESHOLDING

 No description has been provided for this image

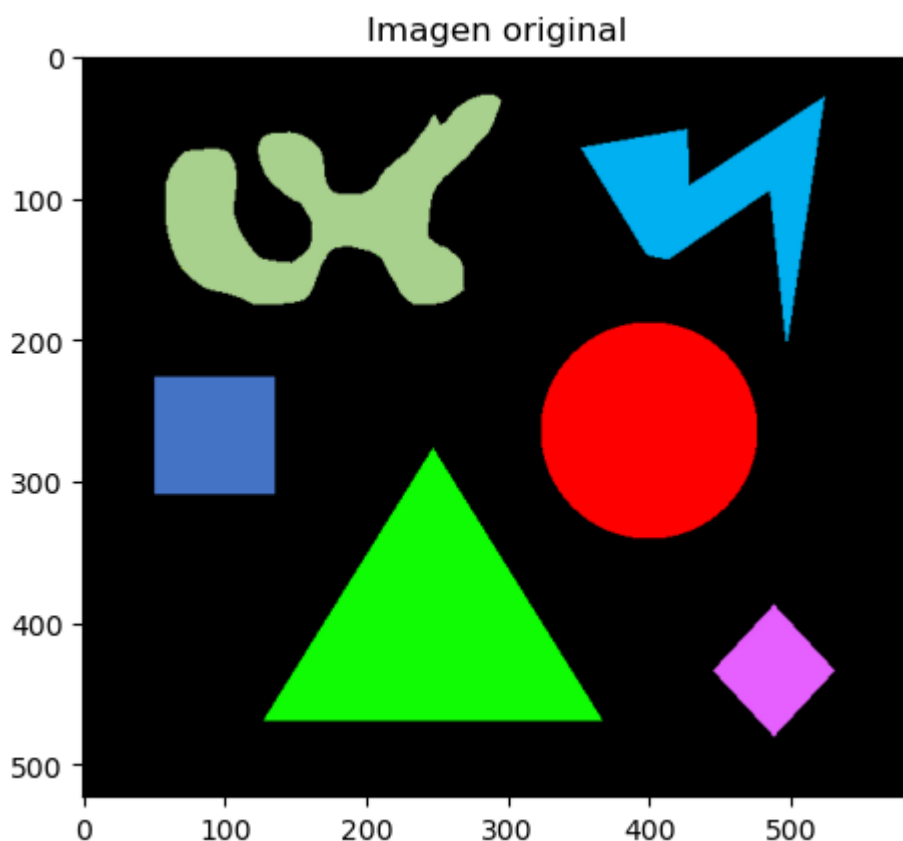
Nombre: Steven Chuiza

Fecha: 21/05/2026

Clase 1 y 2

```
In [2]: import matplotlib.pyplot as plt
import cv2

# Read an image
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb, cmap='gray')
plt.title('Imagen original')
plt.show()
```



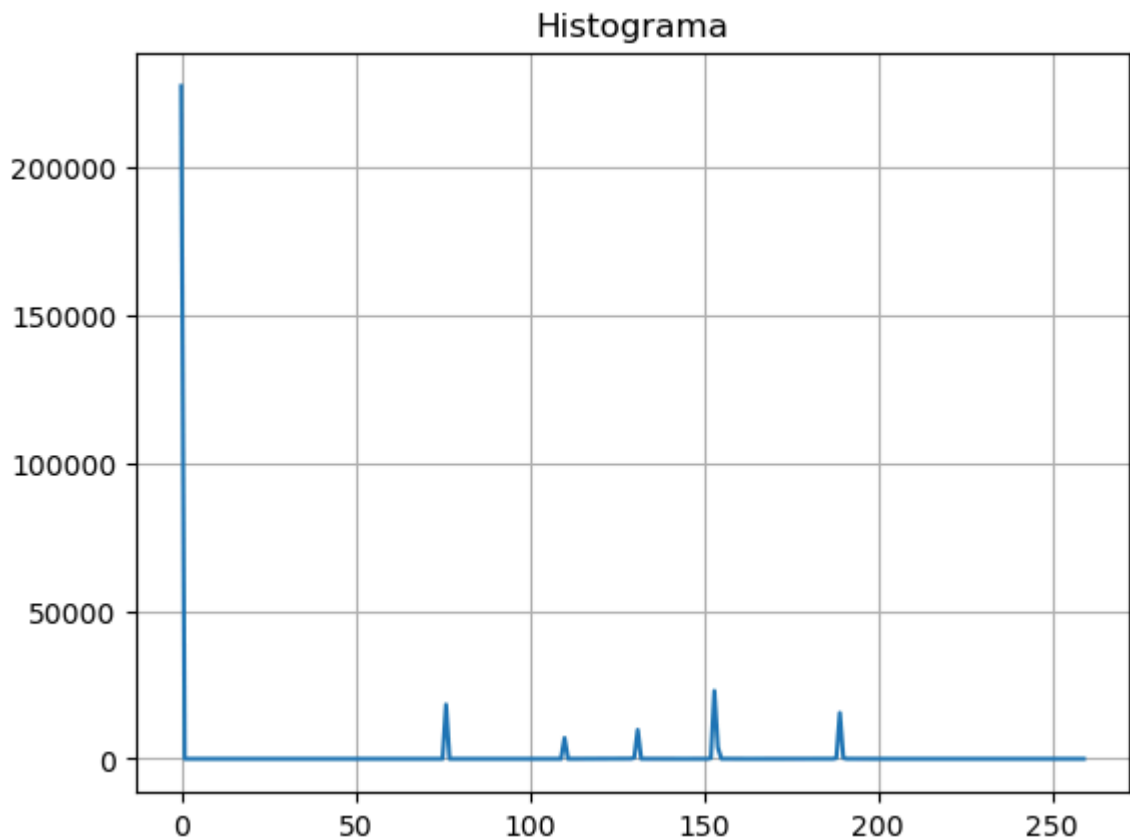
```
In [3]: # Convertimos la imagen a escala de grises
import numpy as np
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
#gray_img = cv2.imread('images/images/figuras.png', 0) # Una forma de leer la im

plt.imshow(gray_img, cmap="gray")
```

```
plt.title('Imagen de grises')  
plt.show()
```



```
In [4]: # Calculamos el histograma  
hist = cv2.calcHist([gray_img], [0], None, [260], [0, 260]) # [0] son los canales  
  
plt.plot(hist)  
plt.grid()  
plt.title('Histograma')  
plt.show()  
  
# Otra forma de mostrar el histograma (solo visualización)  
# plt.hist(gray_img.ravel(), bins=50)  
# plt.grid(True)  
# plt.show()
```



```
In [5]: # Fijamos el umbral en base al histograma anterior
t = 50

# Extraemos la máscara binaria
maxim = 255
_, mask = cv2.threshold(gray_img, t, maxim, cv2.THRESH_BINARY)

# Otra formas de extraer la máscara
# mask = gray_img.copy()
# mask = mask > t

# Visualizamos para corroborar
plt.imshow(mask, cmap='gray')
plt.title('Máscara t=' + str(t))
plt.show()

print(np.unique(mask)) # Atent@s a Los formatos (bool, uint8, etc.)
```

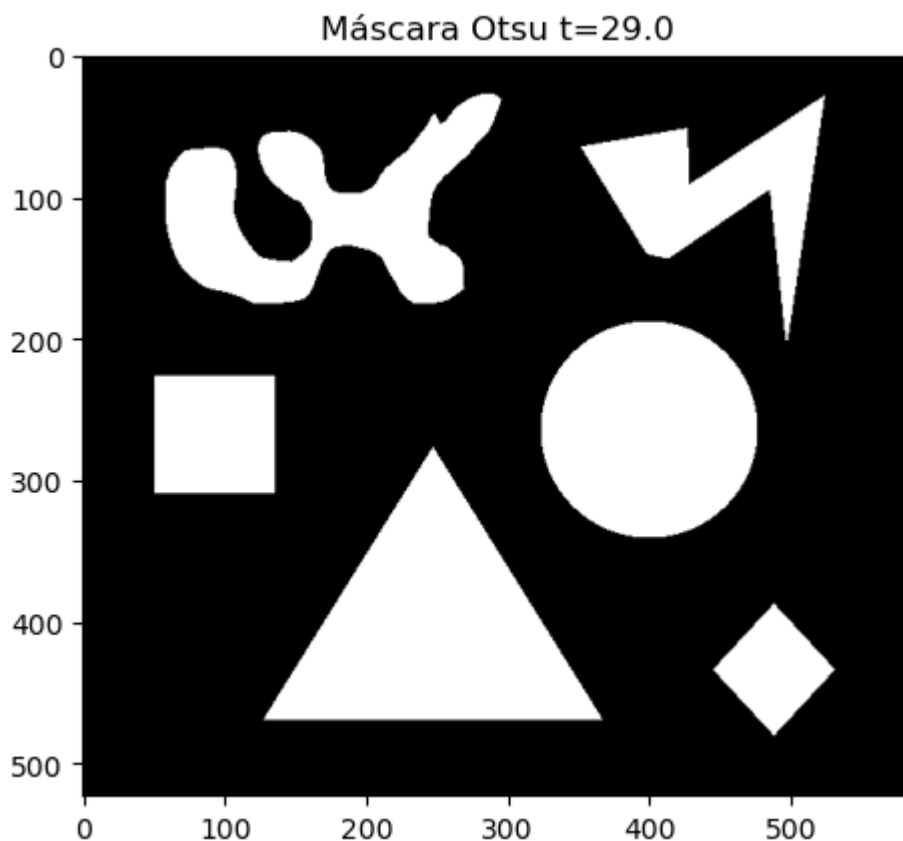


[0 255]

```
In [6]: # Fijamos el umbral con el método de OTSU
t, mask = cv2.threshold(gray_img,0,1,cv2.THRESH_OTSU) # 0 es por defecto y 1 es
print(np.unique(mask))

# Visualizamos para corroborar que se obtiene el mismo resultado
plt.imshow(mask, cmap='gray')
plt.title('Máscara Otsu t=' + str(t))
plt.show()
```

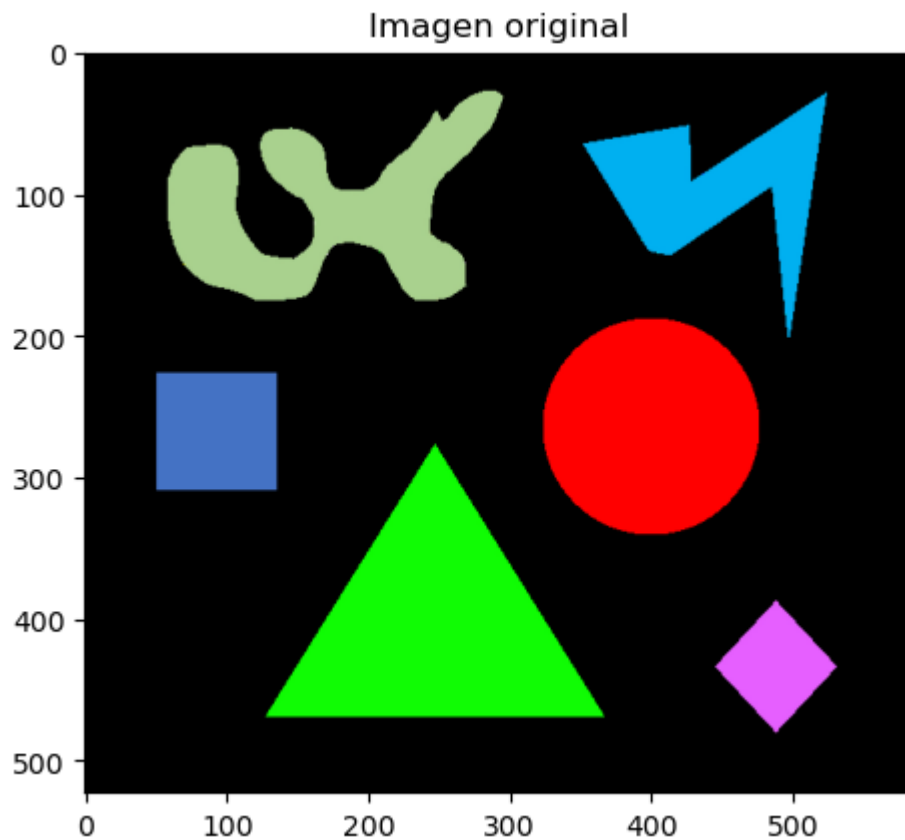
[0 1]



SEGMENTACIÓN MEDIANTE CONVERSIONES DE COLOR

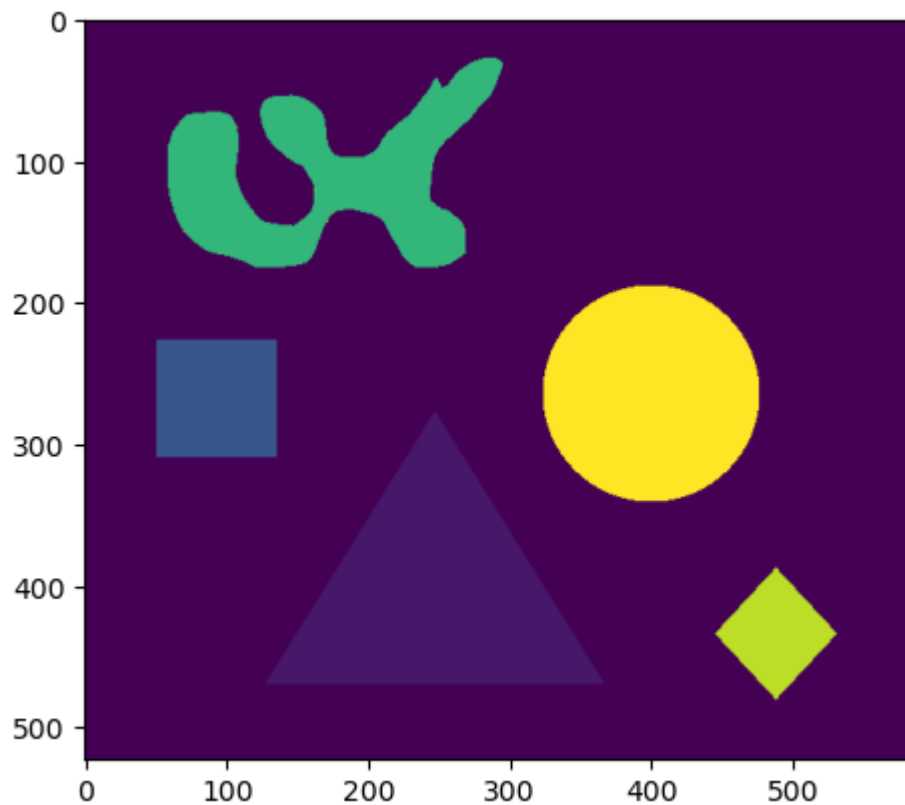
```
In [7]: # Imaginamos que queremos segmentar únicamente el círculo
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb, cmap='gray')
plt.title('Imagen original')
plt.show()
```

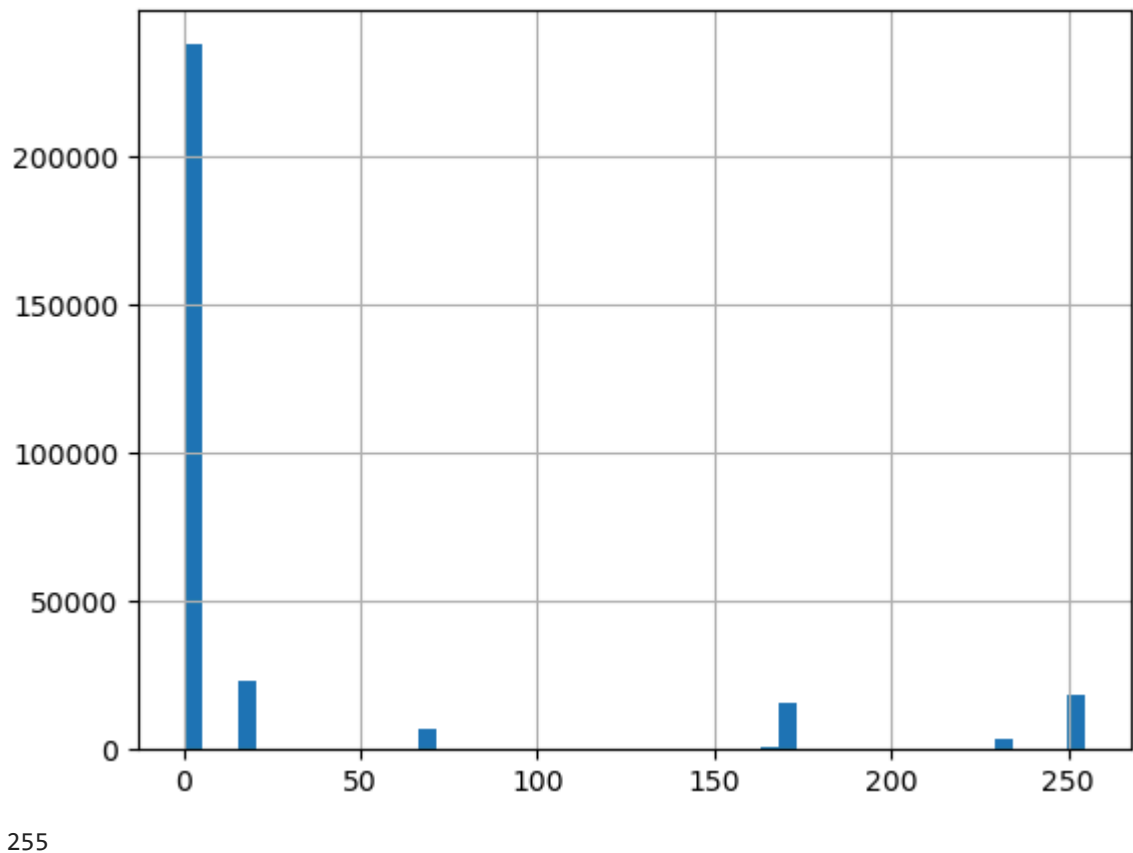


```
In [8]: # Obtenemos el canal rojo en bgr la posición 2 es el color rojo  
R = img[:, :, 2]  
  
plt.imshow(R)
```

Out[8]: <matplotlib.image.AxesImage at 0x1a86cfef610>

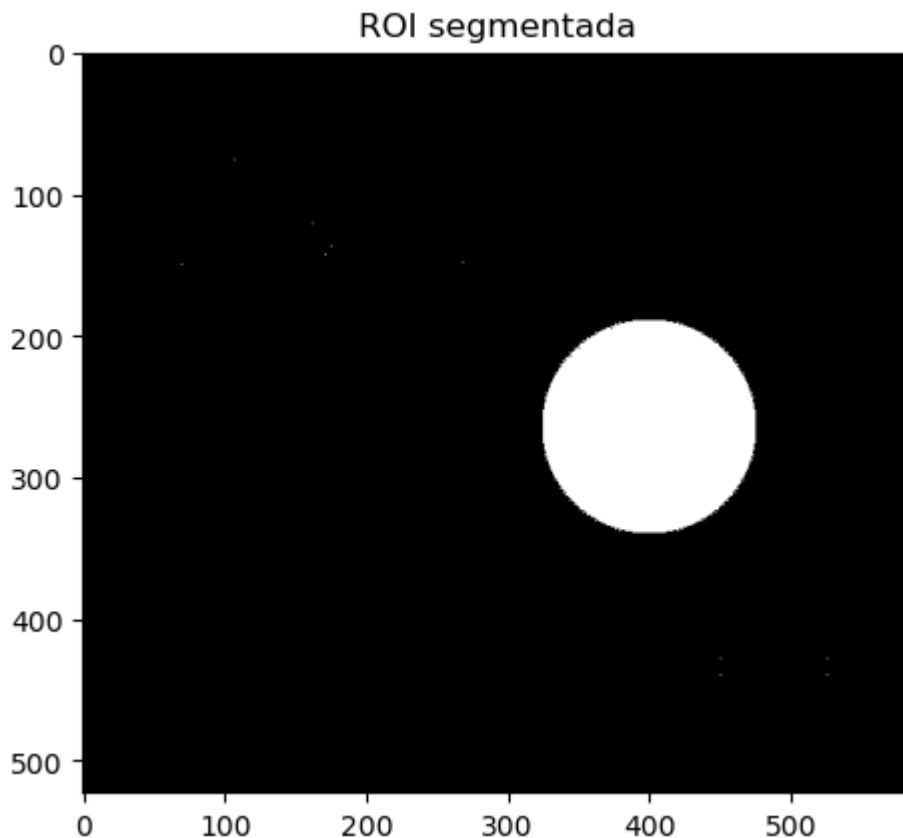


```
In [9]: # Observamos el histograma
plt.hist(R.ravel(), bins=50)
plt.grid(True)
plt.show()
print(np.max(R))
```



```
In [14]: # Fijamos el umbral y extraemos la máscara
t=np.max(R)
mask = R.copy()
mask[mask<t] = 0

plt.imshow(mask, cmap='gray')
plt.title('ROI segmentada')
plt.show()
```



```
In [12]: # Ejercicio individual
# Obtener una máscara con la imagen del rayo azul mediante segmentación por espacio de color
import numpy as np

# Función para extraer los colores secundarios
def convert_to_CMYK(rgb_p):
    with np.errstate(invalid='ignore', divide='ignore'):
        K = 1 - np.max(rgb_p, axis=2)
        C = (1-rgb_p[:, :, 0] - K)/(1-K)
        M = (1-rgb_p[:, :, 1] - K)/(1-K)
        Y = (1-rgb_p[:, :, 2] - K)/(1-K)
        CMYK = (np.dstack((C, M, Y, K))*255).astype('uint8')
    return CMYK

# Leer la imagen
img = cv2.imread('images/figuras.png')
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
rgb_p = rgb.astype('uint8')/255

# Extraer los colores secundarios
CMYK = convert_to_CMYK(rgb_p)

# Extraer los canales por separado
C, M, Y, K = cv2.split(CMYK)
C = C.astype('uint8')

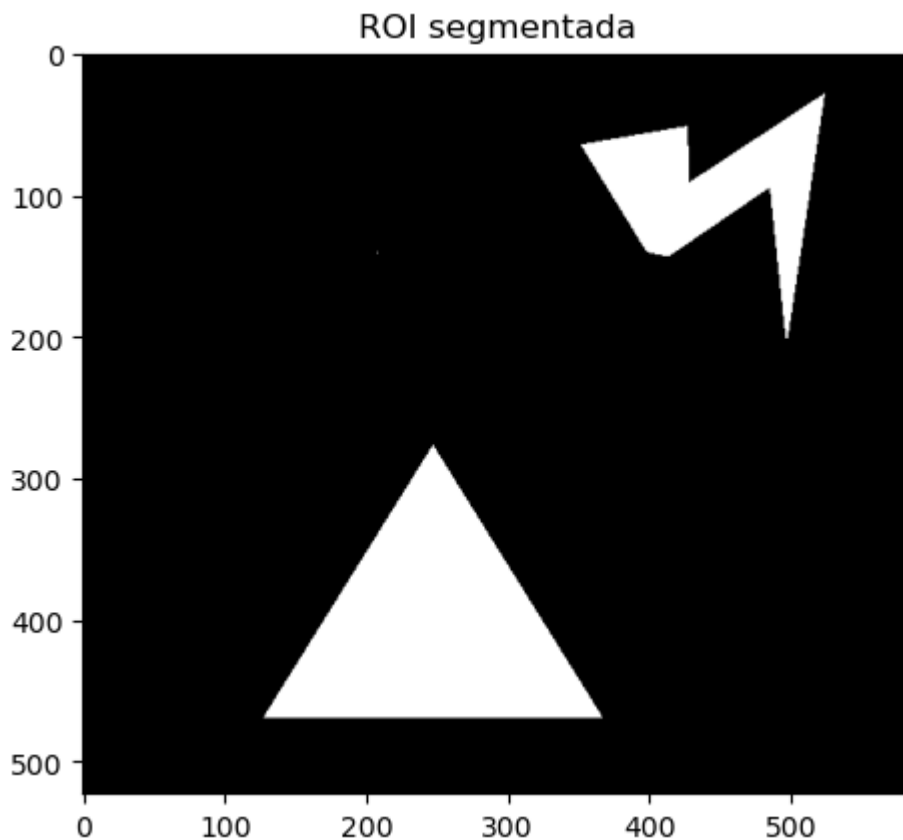
# Calcular la máscara de la figura del rayo
# ???
_, C = cv2.threshold(C, 200, 255, cv2.THRESH_BINARY)

# Visualizar la máscara
plt.imshow(C, cmap='gray')
```



```
plt.title('ROI segmentada')
plt.show()
```

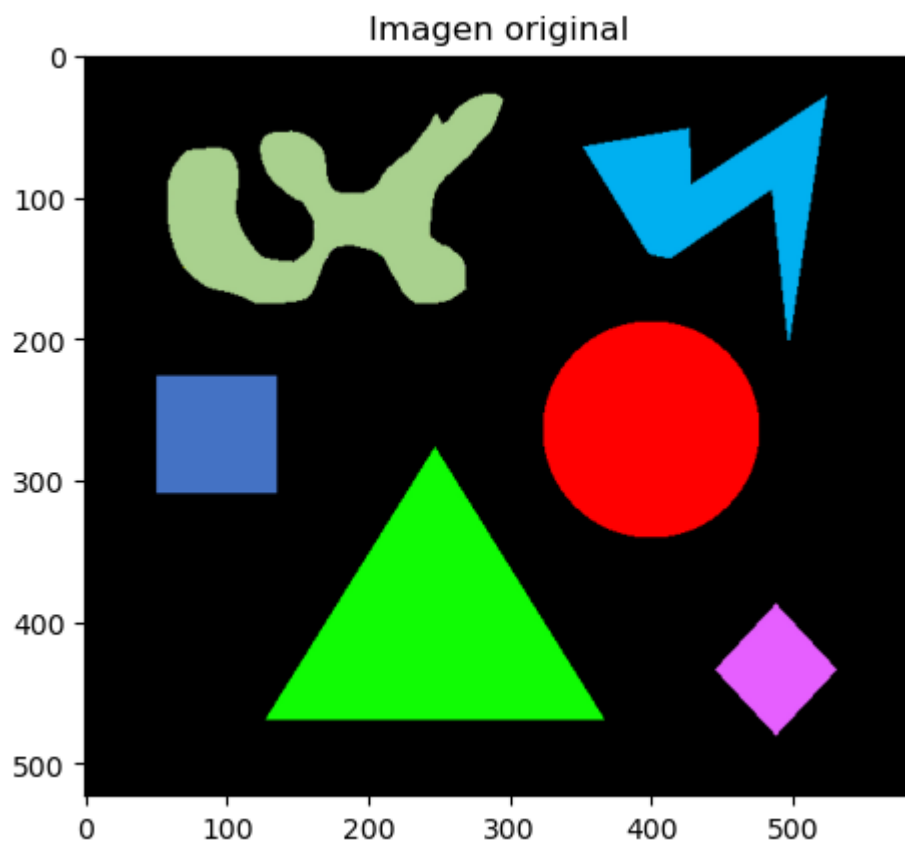
C:\Users\Admin\AppData\Local\Temp\ipykernel_13172\368786131.py:12: RuntimeWarning: invalid value encountered in cast
 CMYK = (np.dstack((C,M,Y,K))*255).astype('uint8')



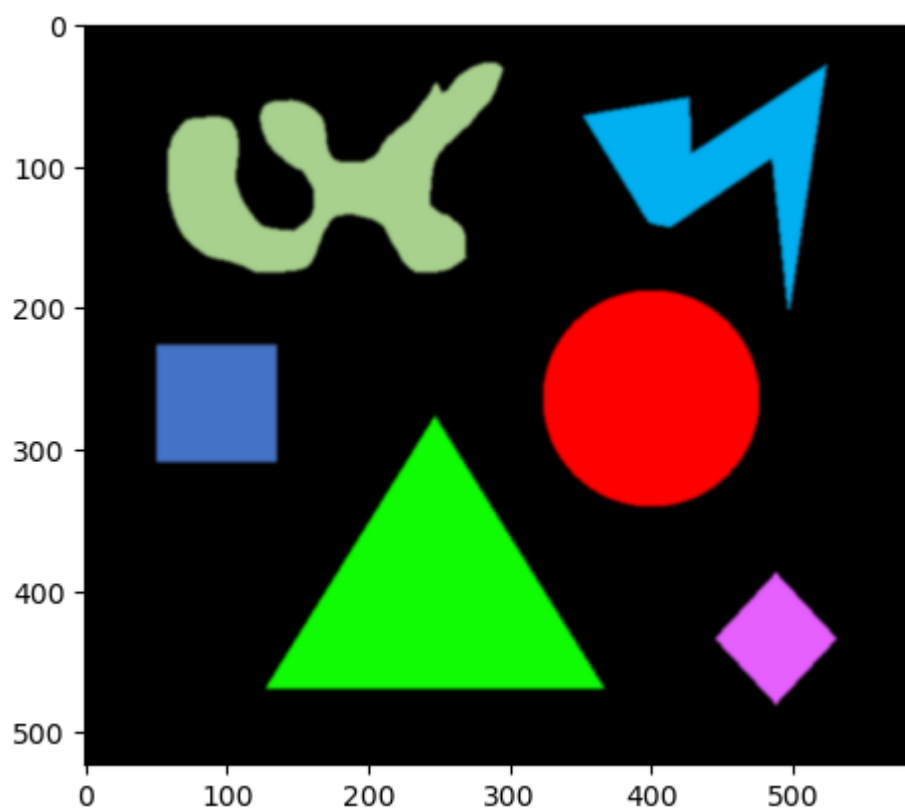
SEGMENTACIÓN SEMÁNTICA CON DETECCIÓN DE BORDES

```
In [15]: # SEGMENTACIÓN SEMÁNTICA CON DETECCIÓN DE BORDES
img = cv2.imread('images/figuras.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img_rgb)
plt.title('Imagen original')
plt.show()
```



```
In [16]: # Aplicamos un filtro gaussiano para emborronar las altas frecuencias
img_gaus = cv2.GaussianBlur(img_rgb, (5,5), 0) # (5x5) es el tamaño del filtro y
plt.imshow(img_gaus, cmap='gray')
plt.show()
```

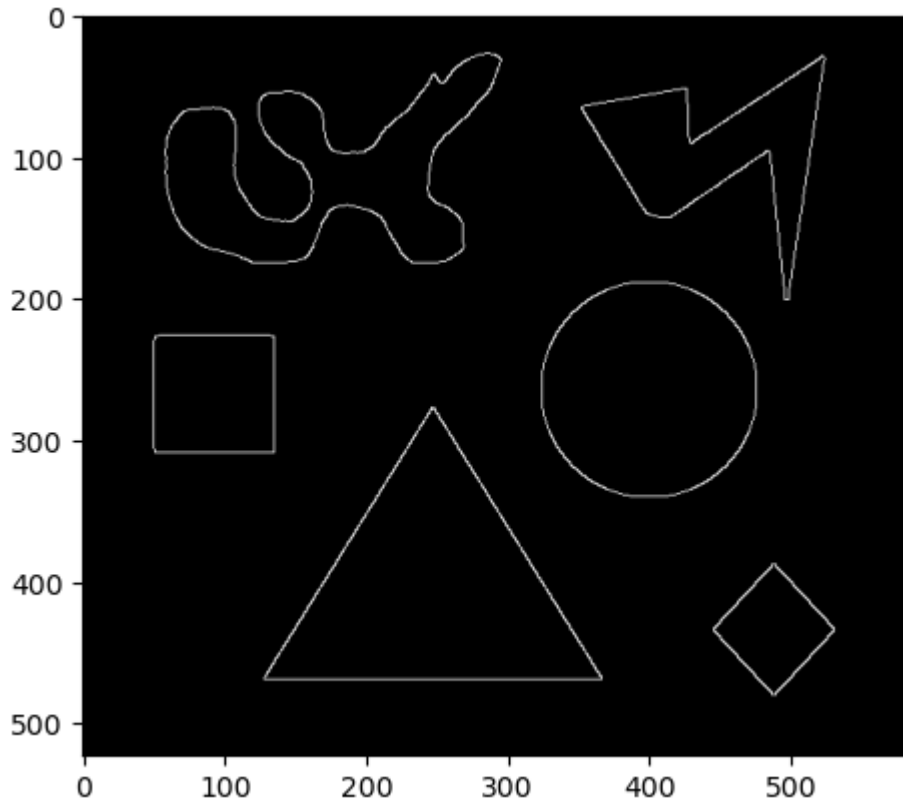


```
In [18]: # Detección de bordes de Canny
canny = cv2.Canny(img_gaus, 255/3, 255) # minValue and maxValue. Canny recomienda
kernel = np.ones((2,2), 'uint8')
```

```
#canny = cv2.dilate(canny, kernel, iterations=2)
#canny = cv2.erode(canny, kernel, iterations=2)
print(kernel)

plt.imshow(canny, cmap='gray')
plt.show()
```

```
[[1 1]
 [1 1]]
```

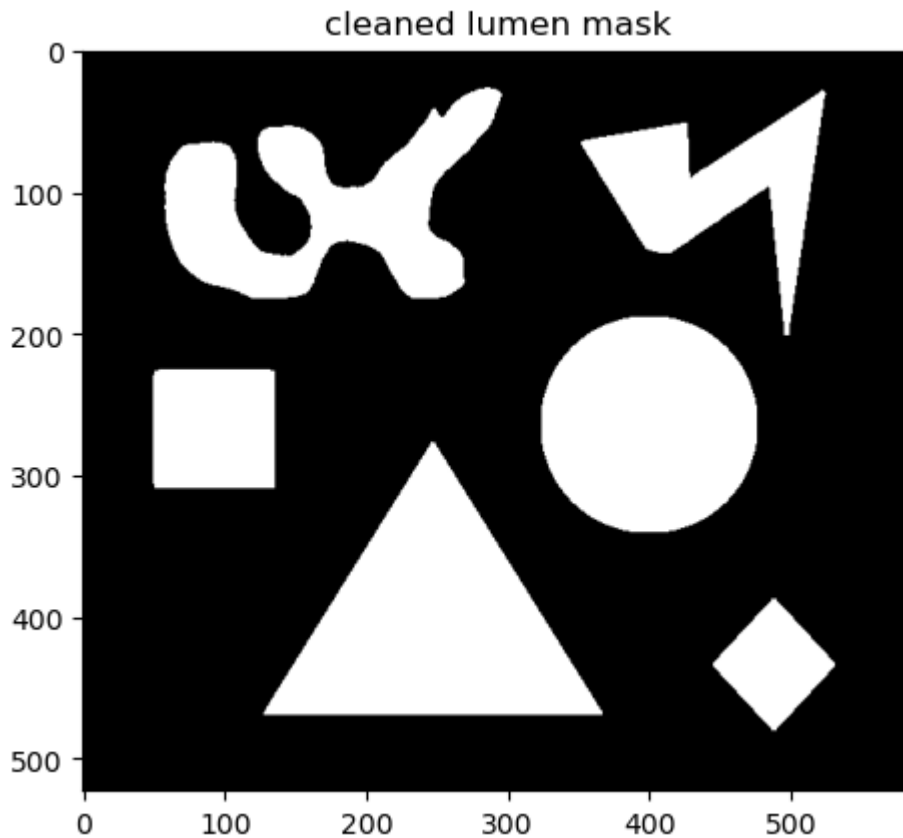


```
In [19]: # Rellenar objetos (solo rellena si no están en contacto con el límite de la ima
from scipy.ndimage.morphology import binary_fill_holes as bfh
bordes = canny.copy()
filled_bordes = bfh(bordes)

plt.imshow(filled_bordes.astype('uint8'), cmap='gray')
plt.title('cleaned lumen mask')
plt.show()
```

C:\Users\Admin\AppData\Local\Temp\ipykernel_13172\2220132281.py:2: DeprecationWarning: Please import `binary\_fill\_holes` from the `scipy.ndimage` namespace; the `scipy.ndimage.morphology` namespace is deprecated and will be removed in SciPy 2.0.0.

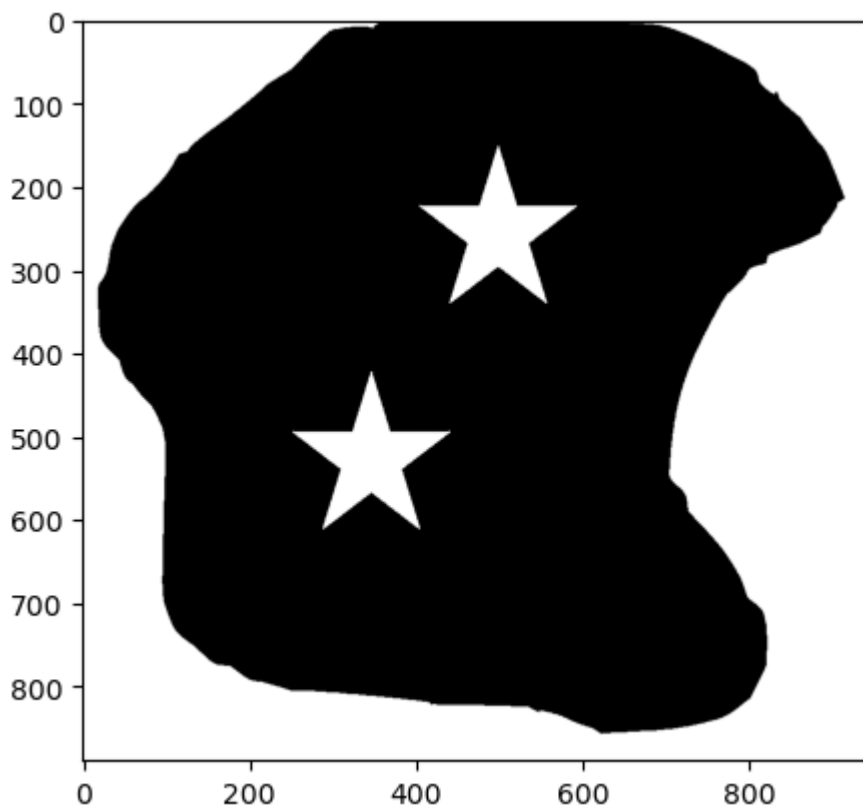
```
from scipy.ndimage.morphology import binary_fill_holes as bfh
```



SEGMENTACIÓN POR CRECIMIENTO DE REGIONES (region growing)

```
In [20]: # Cargar imagen
from skimage import io
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = cv2.imread('images/estrellas.png', 0)

plt.imshow(img, cmap='gray')
plt.show()
```

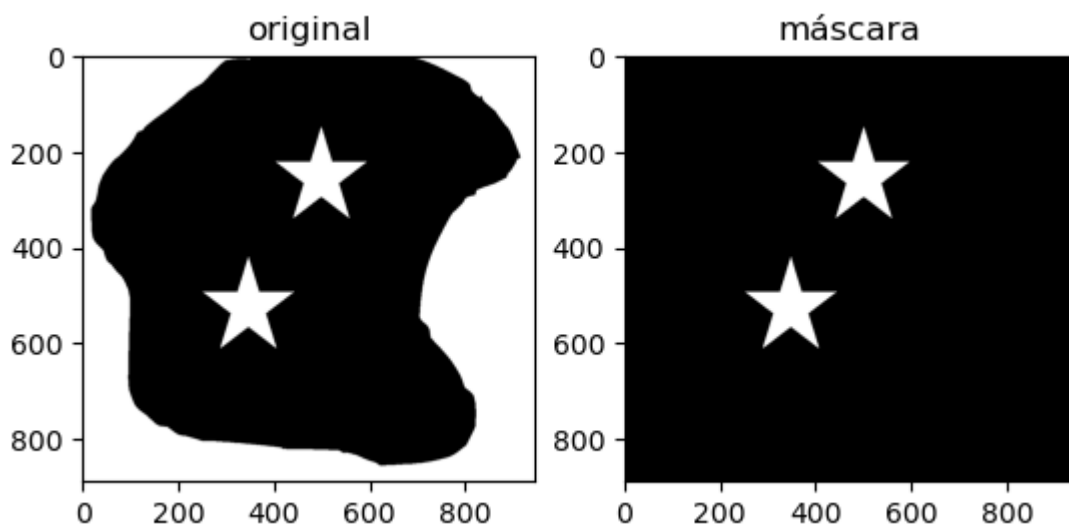


```
In [21]: # Crear una copia de la imagen
mask = img.copy()

# Definir una imagen negra de referencia
h, w = img.shape
ref = np.zeros((h+2, w+2), np.uint8)

# Fijar la semilla. En este caso en la esquina superior izquierda (posición 0,0)
cv2.floodFill(mask, ref, (0,0), 0); # Como nos interesa rellenar de negro, utili

fig, ax = plt.subplots(1,2)
ax[0].imshow(img, cmap='gray'), ax[0].set_title('original')
ax[1].imshow(mask, cmap='gray'), ax[1].set_title('máscara')
plt.show()
```



Repositorio de

GitHub:https://github.com/tiven29/Fundamentos_Machine_Learning/tree/main/.ipynb_check



In []: